

Java'ya Giriş ve İlk Program

Bilgisayar ve Programlamaya Giriş

Program

Araçlar

İlk program

Derleme

Hatalar

Sercan KÜLCÜ · Giresun Üniversitesi

Öğrenme Çıktıları

Program

Programı veri ve işlemler bütünü olarak açıklamak.

Java

Kaynak kod, bytecode ve JVM ilişkisini kurmak.

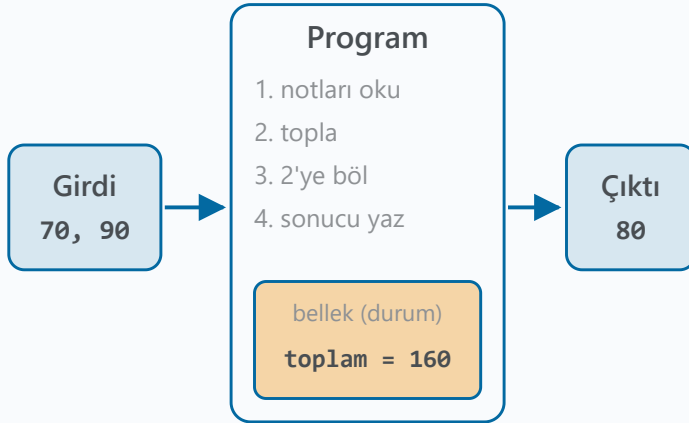
İlk program

Java dosyasını yazıp derlemek ve çalıştırmak.

Hatalar

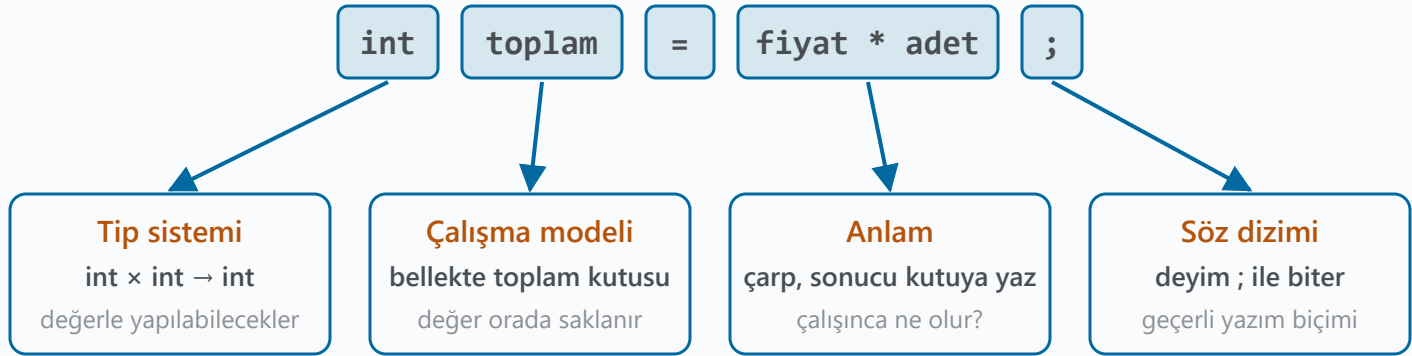
Söz dizimi, çalışma zamanı ve mantık hatalarını ayırt etmek.

Program Nedir?



- Bilgisayarın belirli bir görevi yerine getirmesi için yürüttüğü komutlar bütünüdür.
- Girdi alabilir, veriyi işler, **durum** saklar (bellekteki değerler) ve çıktı üretir.
- Davranışı dilin kuralları ve yazılan algoritma belirler.
- Örnek: iki notun ortalaması → girdi 70, 90; çıktı 80.

Programlama Dilinin Yönleri



- **Söz dizimi** geçerli biçimi, **anlam** çalıştırıldığında ne olacağını tanımlar.
- **Tip sistemi** işlemleri sınırlar; **çalışma modeli** belleğin ve komutların nasıl yürütüldüğünü belirler.
- Derleyici söz dizimini ve tipleri çalıştırmadan önce denetler.

Programlama Paradigmaları

- Buyurgan yaklaşım (imperative) işlemlerin nasıl yapılacağını adım adım belirtir.
- Nesne yönelimli yaklaşım veri ve davranışı nesnelerde birleştirir.
- Fonksiyonel yaklaşım dönüşümleri ve değiştirilemez veriyi öne çıkarır.
- Java birden fazla yaklaşımı birlikte kullanabilir.

Aynı görev, üç yaklaşım: notlar = {70, 90, 80} dizisinin toplamı (240)

Buyurgan

```
int toplam = 0;  
for (int n : notlar)  
    toplam += n;
```

toplam: 0 → 70 → 160 → 240

durum adım adım değişir

Nesne yönelimli

ogrenci : Ogrenci

notlar = {70, 90, 80}
toplam() → 240

ogrenci.toplam()

veri ve davranış bir arada

Fonksiyonel

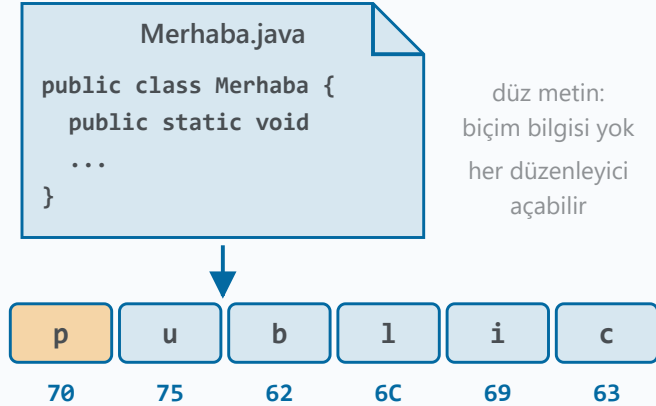
{70, 90, 80}

↓ topla

240

Arrays.stream(notlar).sum()
dönüşüm; dizi değişmez

Düz Metin Kaynak Dosyası



ASCII karakterler UTF-8'de 1 bayt (onaltılık); ç, ğ, ş gibi harfler 2 bayt

- .java dosyası düz metindir; Not Defteri, VS Code veya IDE ile açılabilir.
- Sürüm kontrol sistemleri satır değişikliklerini izleyebilir.
- Karakter kodlaması (UTF-8), satır sonu ve dosya adı tutarlı olmalıdır: Türkçe karakterler yanlış kodlamada bozulur.

Programlama Araçları

Düzenleyici

Kaynak kodu yazma.

IDE

Proje, tamamlama, derleme ve hata ayıklama.

javac

Kaynak kodu bytecode'a derleme.

java

Sınıfı JVM üzerinde çalıştırma.

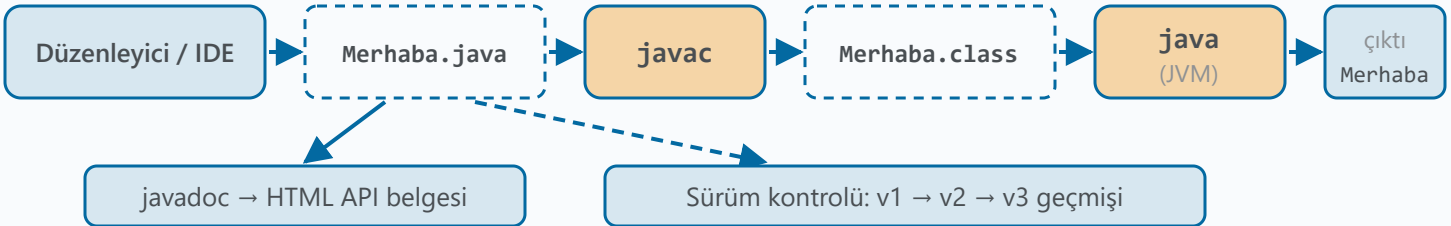
Javadoc

API belgesi üretme.

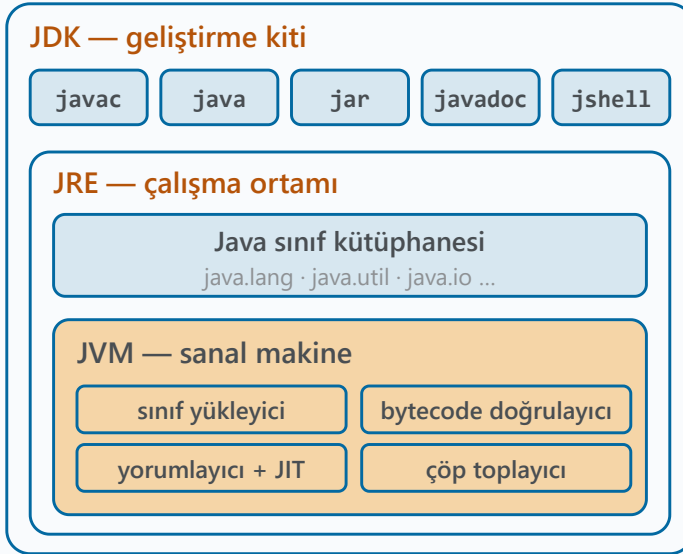
Sürüm kontrolü

Değişiklik geçmişi.

Araçlar bir Java programının yolculuğunda



JDK, JRE ve JVM



- **JVM** (Java Sanal Makinesi) bytecode'u yürüten sanal makinedir.
- **JRE** = JVM + sınıf kütüphanesi: programı çalıştırmak için gerekenler.
- **JDK** = JRE + geliştirme araçları (`javac`, `jar`, `javadoc`...). Geliştirici JDK kurar.
- Oracle, Java 11'den beri ayrı JRE sunmaz; geliştirici JDK kurar (bazı dağıtıcılar JRE'yi ayrıca sunar).

İlk Java Programı

```
public class Merhaba {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Merhaba");
```

```
    }
```

```
}
```

--- sınıf tanımlaması

ad = dosya adı: Merhaba.java

--- main metodu

programın giriş noktası

--- deyim

konsola yaz; ; ile biter

--- } main bloğu biter

--- } sınıf bloğu biter

public

her yerden
erişilebilir

static

nesne oluşturmadan
çağrılır

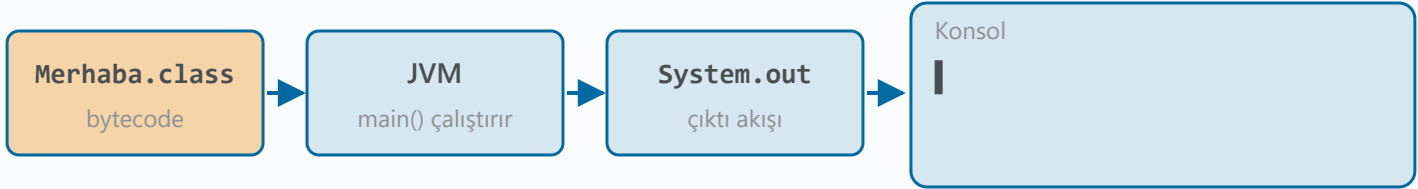
void

değer döndürmez

String[] args

komut satırı
argümanları

Kod İzleme: Merhaba



`java Merhaba` komutuyla JVM başlar ve **Merhaba** sınıfını (Merhaba.class) belleğe yükler.

Adım 1 / 6

```
▶ public class Merhaba {  
2   public static void main(String[] args) {  
3       System.out.println("Merhaba");  
4   }  
5 }
```

DEĞİŞKENLER

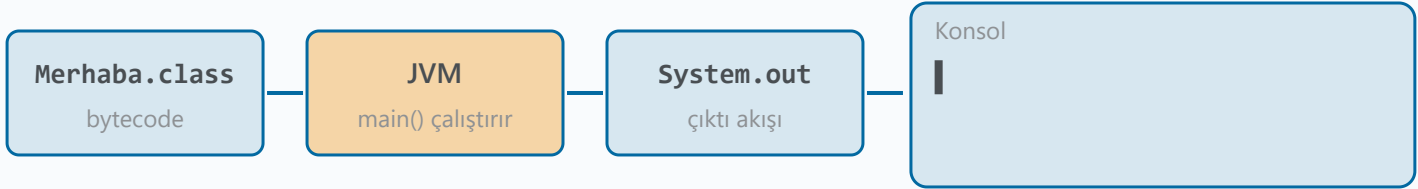
—

ÇAĞRI YIĞINI

boş

ÇIKTI

Kod İzleme: Merhaba



JVM **main** metodunu çağırır: yığında main çerçevesi açılır. Komut satırında argüman verilmediği için args boştur.

Adım 2 / 6

```
1 public class Merhaba {  
2     ▶ public static void main(String[] args) {  
3         System.out.println("Merhaba");  
4     }  
5 }
```

DEĞİŞKENLER

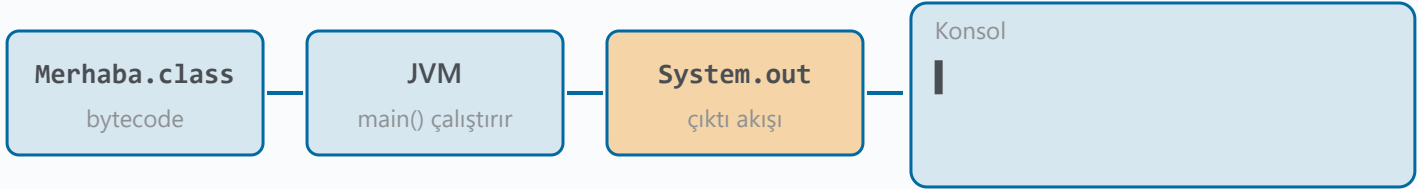
args = boş (argüman yok)

ÇAĞRI YIĞINI

main(args)

ÇIKTI

Kod İzleme: Merhaba



Deyim çalışır: `"Merhaba"` metni `System.out.println` metoduna gönderilir.

Adım 3 / 6

```
1 public class Merhaba {  
2     public static void main(String[] args) {  
3         System.out.println("Merhaba");  
4     }  
5 }
```

DEĞİŞKENLER

`args = boş (argüman yok)`

ÇAĞRI YİĞİNİ

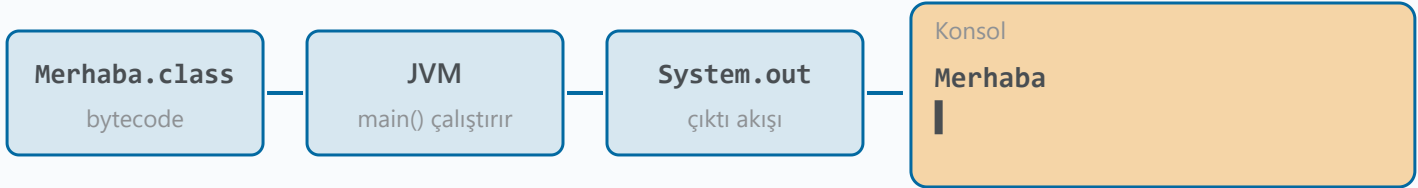
`println("Merhaba")`

`main(args)`

ÇIKTI



Kod İzleme: Merhaba



println metni yazar ve **satır sonu** ekler: imleç alt satıra geçer. Metot biter, main'e dönlür.

Adım 4 / 6

```
1 public class Merhaba {  
2     public static void main(String[] args) {  
3         System.out.println("Merhaba");  
4     }  
5 }
```

DEĞİŞKENLER

args = boş (argüman yok)

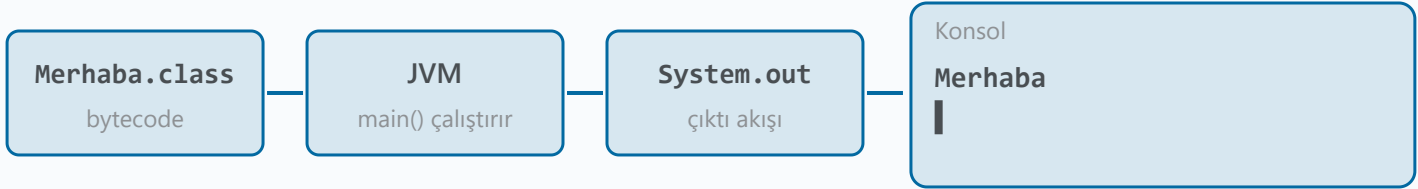
ÇAĞRI YİĞİNİ

main(args)

ÇIKTI

Merhaba

Kod İzleme: Merhaba



main bloğunun `}` karakterine gelindi: main biter, çerçevesi yığından kaldırılır.

Adım 5 / 6

```
1 public class Merhaba {  
2     public static void main(String[] args) {  
3         System.out.println("Merhaba");  
4     }  
5 }
```

DEĞİŞKENLER

args = boş (argüman yok)

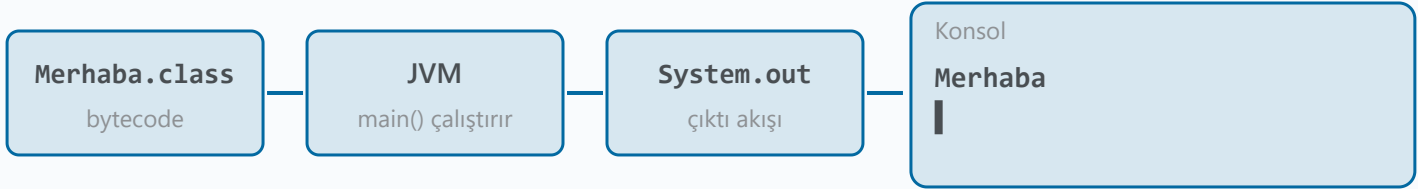
ÇAĞRI YIĞINI

boş

ÇIKTI

Merhaba

Kod İzleme: Merhaba



Çalışacak başka kod yok: JVM kapanır, program **sonlanır**. Toplam 1 deyim çalıştı, 1 satır çıktı üretildi.

Adım 6 / 6

```
1 public class Merhaba {  
2     public static void main(String[] args) {  
3         System.out.println("Merhaba");  
4     }  
▶ }
```

DEĞİŞKENLER

—

ÇAĞRI YIĞINI

boş

ÇIKTI

Merhaba

Kod İzleme: `print` ve `println`

Konsol

|

imleç: |
satır 1, sütun 1

JVM **main** metodunu çağırır; konsol henüz boş, imleç 1. satırın başında.

Adım 1 / 5

```
1 public class Selam {  
2     public static void main(String[] args) {  
3         System.out.print("Merhaba ");  
4         System.out.println("Dünya");  
5         System.out.println("Java");  
6     }  
7 }
```

ÇAĞRI YIĞINI

main(args)

ÇIKTI

Kod İzleme: `print` ve `println`

Konsol

Merhaba |

imleç: |

satır 1, sütun 9

print "Merhaba " yazar ama **satır sonu eklemmez**: imleç aynı satırda, boşluğun hemen arkasında kalır.

Adım 2 / 5

```
1 public class Selam {  
2     public static void main(String[] args) {  
3         System.out.print("Merhaba ");  
4         System.out.println("Dünya");  
5         System.out.println("Java");  
6     }  
7 }
```

ÇAĞRI YIĞINI

main(args)

ÇIKTI

Merhaba

Kod İzleme: `print` ve `println`

Konsol

Merhaba Dünya



imleç: 
satır 2, sütun 1

println "Dünya" yazar ve **satır sonu** ekler: sonraki çıktı alt satırdan başlar.

Adım 3 / 5

```
1 public class Selam {  
2     public static void main(String[] args) {  
3         System.out.print("Merhaba ");  
4         System.out.println("Dünya");  
5         System.out.println("Java");  
6     }  
7 }
```

ÇAĞRI YIĞINI

main(args)

ÇIKTI

Merhaba Dünya

Kod İzleme: `print` ve `println`

Konsol

Merhaba Dünya

Java

|

imleç: |
satır 3, sütun 1

`println` "Java" yazar ve **satır sonu** ekler: sonraki çıktı alt satırdan başlar.

Adım 4 / 5

```
1 public class Selam {  
2     public static void main(String[] args) {  
3         System.out.print("Merhaba ");  
4         System.out.println("Dünya");  
5         System.out.println("Java");  
6     }  
7 }
```

ÇAĞRI YIĞINI

main(args)

ÇIKTI

Merhaba Dünya
Java

Kod İzleme: print ve println

Konsol

Merhaba Dünya

Java

|

imleç: |
satır 3, sütun 1

main biter; konsolda **2 satır** var: 3 çağrı yapıldı ama yalnızca println satırı bitirdi.

Adım 5 / 5

```
1 public class Selam {  
2     public static void main(String[] args) {  
3         System.out.print("Merhaba ");  
4         System.out.println("Dünya");  
5         System.out.println("Java");  
6     }  
7 }
```

ÇAĞRI YIĞINI

boş

ÇIKTI

Merhaba Dünya
Java

Kaçış Dizileri

Dizi	Anlamı	Örnek	Çıktı
<code>\n</code>	yeni satır	<code>"A\nB"</code>	A, alt satırda B
<code>\t</code>	sekme (tab)	<code>"Ad\tNot"</code>	Ad Not
<code>\"</code>	çift tırnak	<code>"\"Java\""</code>	"Java"
<code>\\</code>	ters eğik çizgi	<code>"C:\\ders"</code>	C:\ders
<code>\'</code>	tek tırnak (char içinde)	<code>'\''</code>	'

- Dizgi içinde yazılamayan veya özel anlamı olan karakterler `\` ile başlayan dizilerle yazılır.
- Tırnak içindeki `"` metni bitirir; metnin parçası olacaksa `\"` yazılır.

Kod İzleme: Kaçış Dizileri

Konsol

|

main başlar; konsol boş.

Adım 1 / 5

```
1 public class Kacis {  
▶ public static void main(String[] args) {  
3     System.out.println("Ad\tNot");  
4     System.out.println("Ali\t85");  
5     System.out.println("\t\"Java\" dili");  
6     System.out.print("Satır 1\nSatır 2\n");  
7 }  
8 }
```

ÇAĞRI YIĞINI

main(args)

ÇIKTI

Kod İzleme: Kaçış Dizileri

Konsol

Ad

Not

|

`\t` imleci bir sonraki **sekme durağına** (8. sütun) taşır.

Adım 2 / 5

```
1 public class Kacis {  
2     public static void main(String[] args) {  
3         System.out.println("Ad\tNot");  
4         System.out.println("Ali\t85");  
5         System.out.println("\t\"Java\" dili");  
6         System.out.print("Satır 1\nSatır 2\n");  
7     }  
8 }
```

ÇAĞRI YİĞİNİ

main(args)

ÇIKTI

Ad

Not

Kod İzleme: Kaçış Dizileri

Konsol

Ad	Not
Ali	85

Sekme sayesinde 85, Not başlığının altına hizalanır: basit tablo.

Adım 3 / 5

```
1 public class Kacis {  
2     public static void main(String[] args) {  
3         System.out.println("Ad\tNot");  
4         System.out.println("Ali\t85");  
5         System.out.println("\n\"Java\" dili");  
6         System.out.print("Satır 1\nSatır 2\n");  
7     }  
8 }
```

ÇAĞRI YİĞİNİ

main(args)

ÇIKTI

Ad	Not
Ali	85

Kod İzleme: Kaçış Dizileri

Konsol

```
Ad      Not
Ali     85
"Java"  dili
|
```

\ " metni bitirmeden çift tırnak yazar.

Adım 4 / 5

```
1 public class Kacis {
2     public static void main(String[] args) {
3         System.out.println("Ad\tNot");
4         System.out.println("Ali\t85");
5         System.out.println("\"Java\" dili");
6         System.out.print("Satır 1\nSatır 2\n");
7     }
8 }
```

ÇAĞRI YİĞİNİ

main(args)

ÇIKTI

```
Ad      Not
Ali     85
"Java"  dili
```

Kod İzleme: Kaçış Dizileri

Konsol

"Java" dili**Satır 1****Satır 2****|**

Tek print iki satır üretti: `\n` satır sonu karakteridir; println de satır sonu ekler (Windows'ta `\r\n`).

Adım 5 / 5

```
1 public class Kacis {  
2     public static void main(String[] args) {  
3         System.out.println("Ad\tNot");  
4         System.out.println("Ali\t85");  
5         System.out.println("\"Java\" dili");  
6         System.out.print("Satır 1\nSatır 2\n");  
7     }  
8 }
```

ÇAĞRI YİĞİNİ

main(args)

ÇIKTI

```
Ad      Not  
Ali     85  
"Java"  dili  
Satır 1  
Satır 2
```

Java Dosyasının Yapısı

```
package okul;  
  
import java.util.Scanner;  
  
public class Program {  
    public static void main(String[] args) {  
        System.out.println("Merhaba");  
    }  
}
```

okul/Program.java

1 · package okul;

2 · import ...;

3 · class Program { ... }

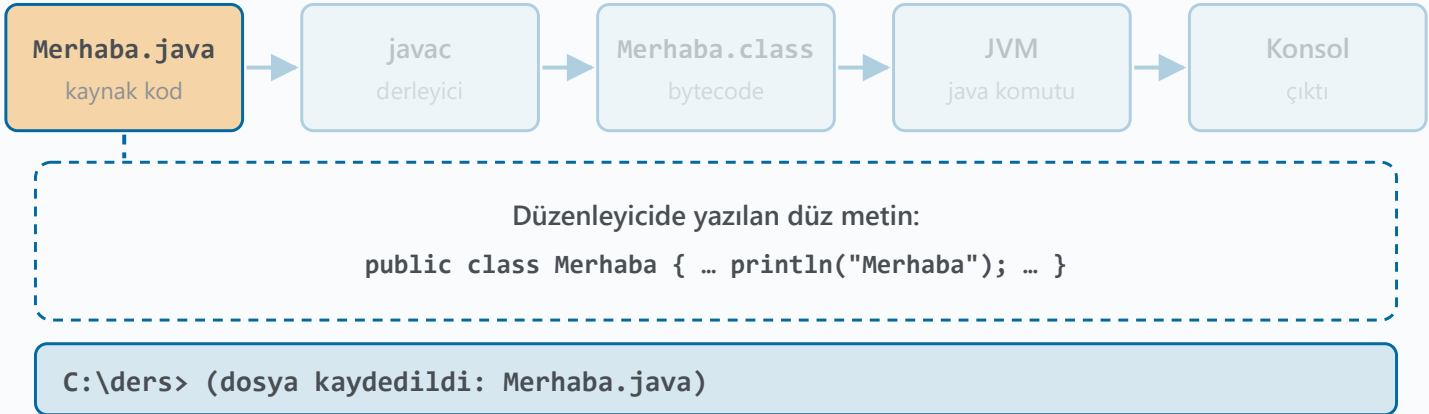
main(...) { ... }

deyim;

deyim;

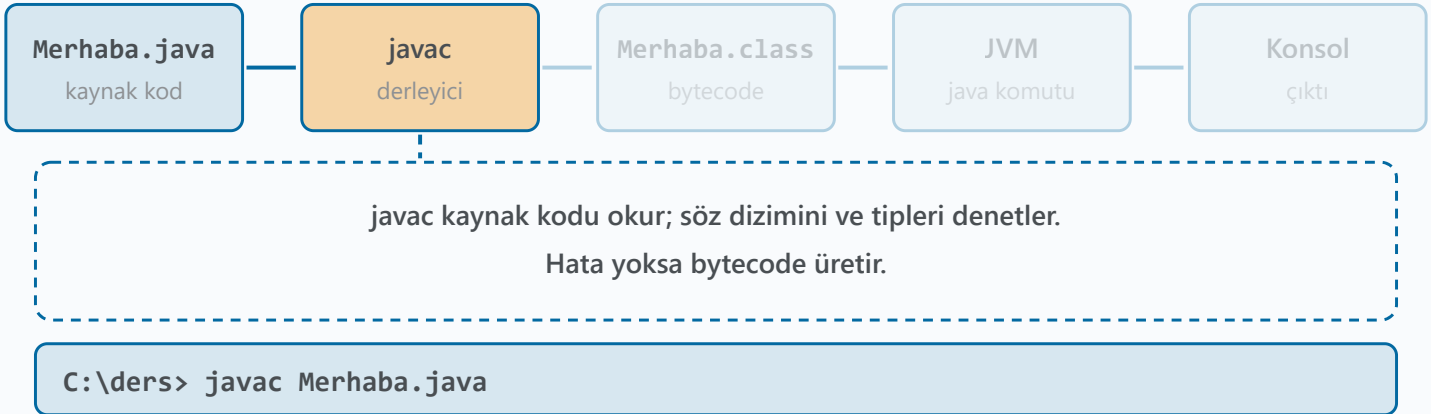
- package ve import tanımlamaları sınıftan önce gelir; public sınıf adı dosya adıyla eşleşir.
- `package okul;` → dosya `okul/` klasöründe durur; çalıştırma: `java okul.Program` .

Derleme Hattı Adım Adım



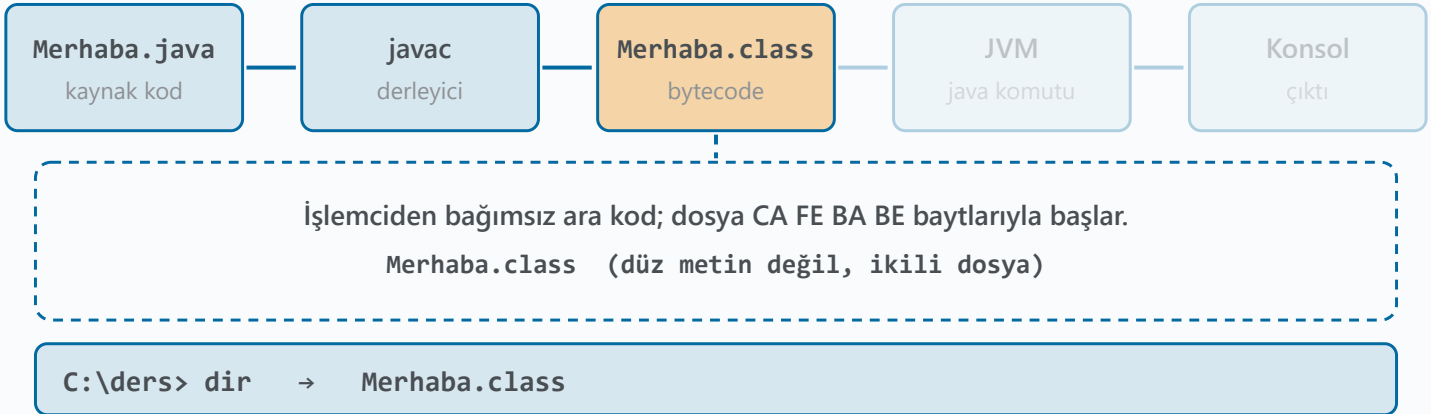
1. Yaz: kaynak kod `Merhaba.java` adıyla kaydedilir. Dosya adı public sınıf adıyla aynı olmalıdır.

Derleme Hattı Adım Adım



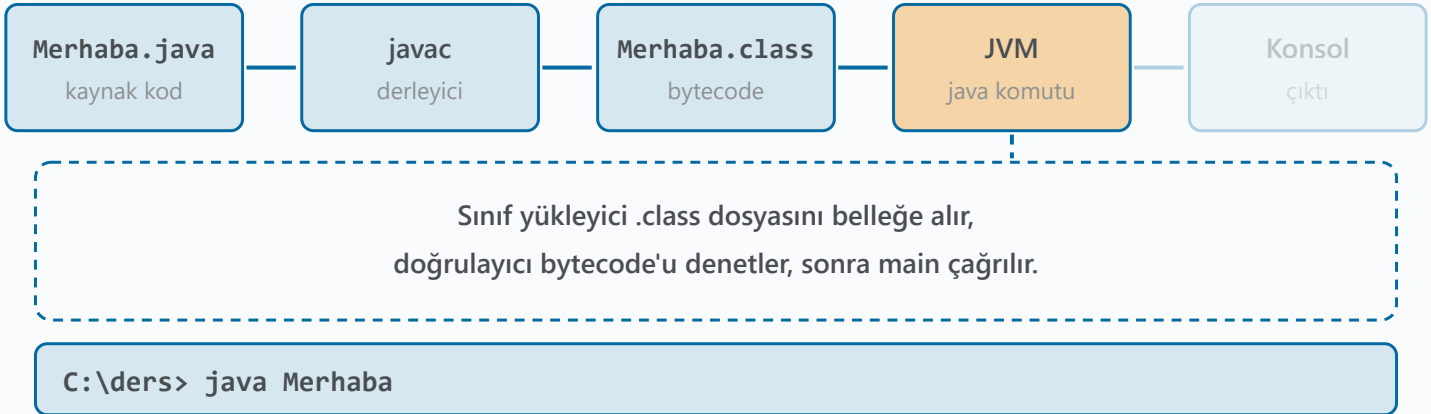
2. Derle: `javac` (JDK'daki derleyici) tüm dosyayı çalıştırmadan önce denetler.

Derleme Hattı Adım Adım



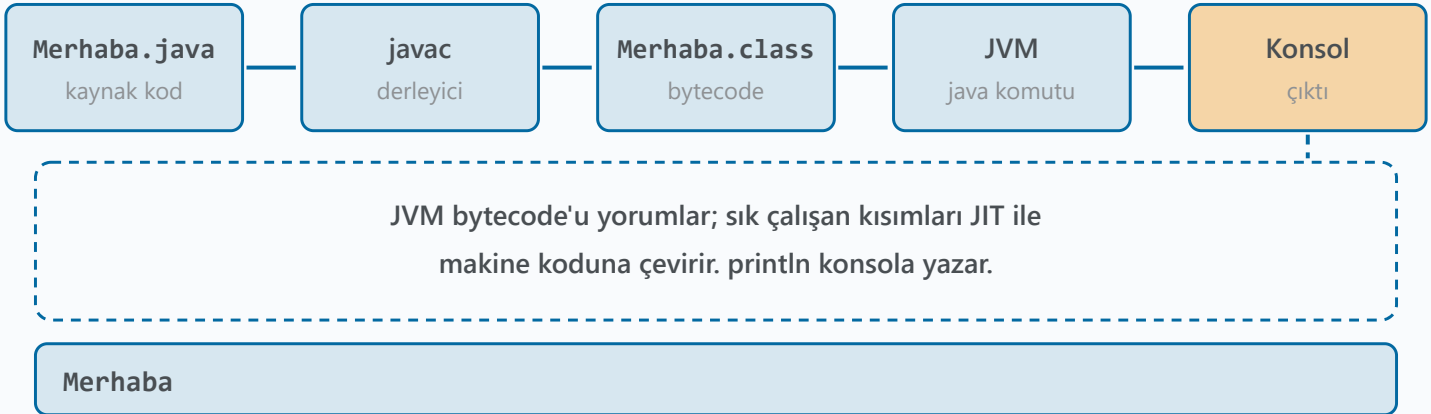
3. Bytecode: derleme başarılıysa aynı klasörde `Merhaba.class` oluşur. Bu dosya Windows, Linux ve macOS'ta aynıdır.

Derleme Hattı Adım Adım



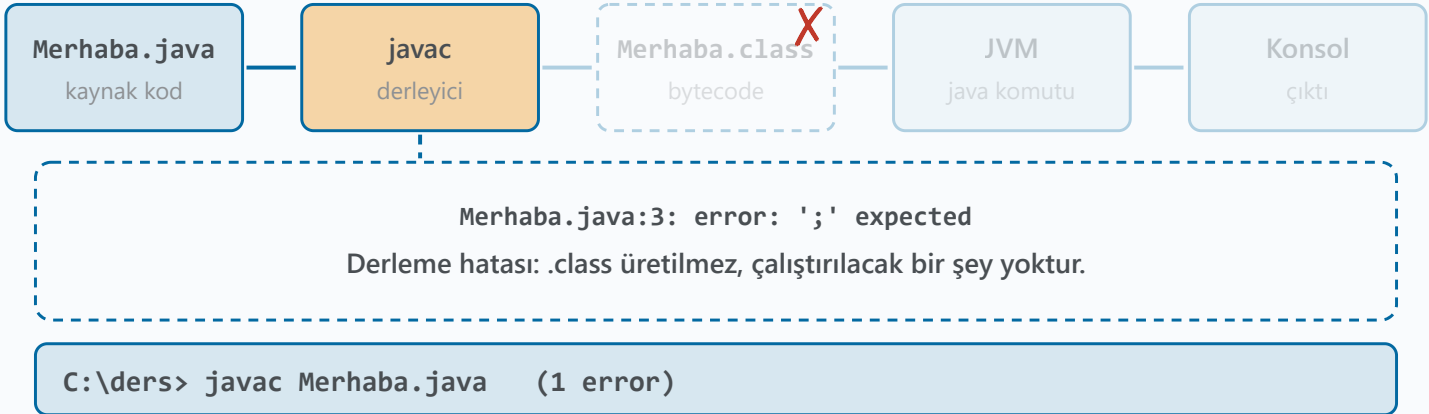
4. Çalıştır: `java Merhaba` JVM'yi başlatır. Sınıf adı yazılır; **.class uzantısı yazılmaz.**

Derleme Hattı Adım Adım



5. Çıktı: konsolda `Merhaba` görünür. Değişiklik yaparsanız yeniden **derlemeniz** gerekir.

Derleme Hattı Adım Adım



Hata olursa: javac satır numarasıyla hata bildirir ve **.class üretmez**. Önce derleme hataları giderilir.

JDK Kurulumu ve Komutlar

Komut	Ne yapar?
<code>java -version</code>	kurulu Java sürümünü gösterir
<code>javac -version</code>	derleyici sürümünü gösterir (javac yalnız JDK içindedir)
<code>javac Merhaba.java</code>	derler: <code>Merhaba.class</code> üretir
<code>java Merhaba</code>	çalıştırır (uzantı yazılmaz)
<code>java Merhaba.java</code>	Java 11+: derle ve çalıştır tek adımda, .class bırakmaz

PATH ve JAVA_HOME

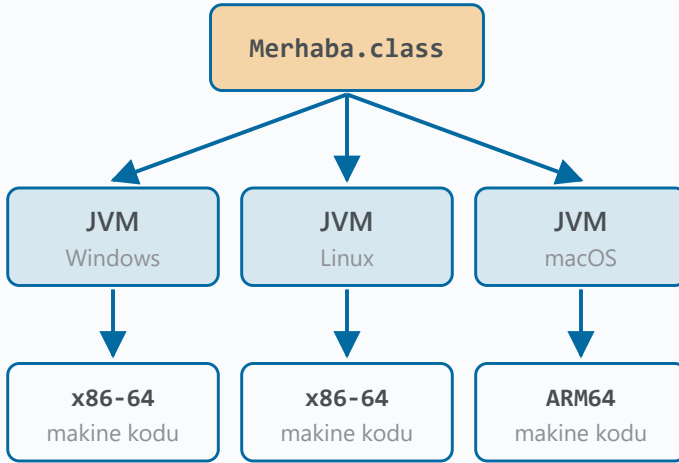
Komutlar bulunamıyorsa JDK'nın `bin` klasörünü PATH ortam değişkenine ekle.

`JAVA_HOME` JDK klasörünü gösterir; araçlar bunu kullanır.

Windows konsolunda Türkçe

Derlerken `javac -encoding UTF-8 Merhaba.java`. Çıktıda harfler bozuksa konsolda önce `chcp 65001` çalıştır.

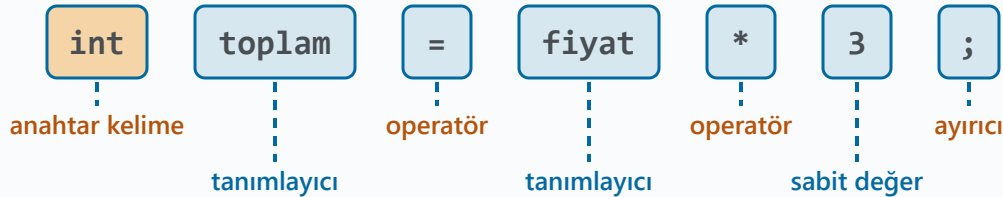
Bytecode ve JVM



bir kez derle, JVM olan her yerde çalıştır

- Derleyici işlemciye özel makine kodu yerine **bytecode** üretir.
- Her platformun kendi JVM'si bytecode'u yorumlar; sık çalışan kısımları JIT ile o işlemcinin makine koduna çevirir.
- Sınıf yükleme, doğrulama ve bellek yönetimi (çöp toplama) çalışma ortamının parçalarıdır.

Temel Programlama Unsurları



- Derleyici kodu önce **belirteçlere** (token) ayırır: anahtar kelime, tanımlayıcı, sabit değer, operatör, ayırıcı.
- Java büyük-küçük harfe duyarlıdır: `sayi` ve `Sayi` farklı adlardır.
- Deyimler çoğunlukla noktalı virgülle biter; bloklar süslü parantezle çevrilir.

İfadeler ve Deyimler

- **İfade** bir değer üretir: `2 + 3` → 5, `sayi > 0` → true.
- **Deyim** bir işlemi tamamlar: `toplam = 5;` veya `System.out.println(toplam);`
- **Blok**, süslü parantez içindeki deyimleri tek yapı olarak gruplar.
- Bir deyimin içinde bir veya daha çok ifade bulunabilir.

blok { ... }

deyim (atama)

toplam

=

fiyat * adet

;

ifade → 360

deyim (atama)

indirim

=

toplam > 300

;

ifade → true

İfade, Deyim ve Blok Adım Adım

Bellek (değişkenler)



fiyat için bellekte int kutusu ayrılır ve 120 yazılır.

Adım 1 / 7

```
► int fiyat = 120;           // tanımlama deyimi
2 int adet = 3;              // tanımlama deyimi
3 int toplam = fiyat * adet; // ifade + atama
4 if (toplam > 300) {         // koşul + blok
5     System.out.println("İndirim");
6 }
```

DEĞİŞKENLER

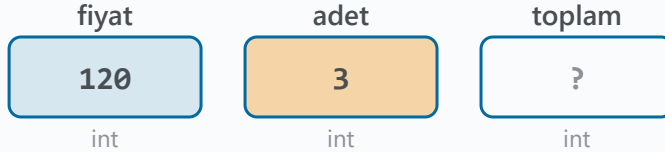
fiyat = 120

ÇIKTI



İfade, Deyim ve Blok Adım Adım

Bellek (değişkenler)



adet kutusu oluşur: 3.

Adım 2 / 7

```
1 int fiyat = 120;           // tanımlama deyimi
2 ► int adet = 3;            // tanımlama deyimi
3 int toplam = fiyat * adet; // ifade + atama
4 if (toplam > 300) {         // koşul + blok
5     System.out.println("İndirim");
6 }
```

DEĞİŞKENLER

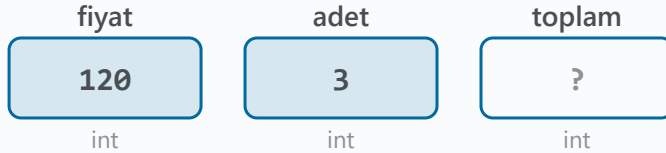
fiyat = 120

adet = 3

ÇIKTI

İfade, Deyim ve Blok Adım Adım

Bellek (değişkenler)



İfade



Önce sağdaki **ifade** değerlendirilir: fiyat * adet → 120 * 3 → **360**.

Adım 3 / 7

```
1 int fiyat = 120;           // tanımlama deyimi
2 int adet = 3;              // tanımlama deyimi
▶ 3 int toplam = fiyat * adet; // ifade + atama
4 if (toplam > 300) {         // koşul + blok
5     System.out.println("İndirim");
6 }
```

DEĞİŞKENLER

fiyat = 120

adet = 3

ÇIKTI



İfade, Deyim ve Blok Adım Adım

Bellek (değişkenler)



Üretilen değer **toplam** kutusuna atanır: toplam = 360.

Adım 4 / 7

```
1 int fiyat = 120;           // tanımlama deyimi
2 int adet = 3;              // tanımlama deyimi
3 ► int toplam = fiyat * adet; // ifade + atama
4 if (toplam > 300) {         // koşul + blok
5     System.out.println("İndirim");
6 }
```

DEĞİŞKENLER

fiyat = 120

adet = 3

toplam = 360

ÇIKTI

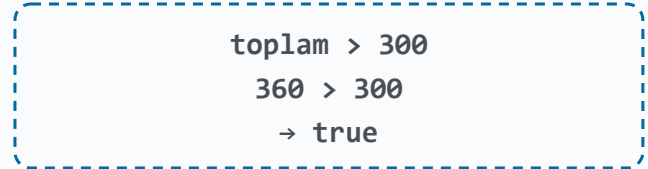


İfade, Deyim ve Blok Adım Adım

Bellek (değişkenler)



İfade



Koşul ifadesi: toplam > 300 → 360 > 300 → **true**. Doğru olduğu için bloğa girilir.

Adım 5 / 7

```
1 int fiyat = 120;           // tanımlama deyimi
2 int adet = 3;              // tanımlama deyimi
3 int toplam = fiyat * adet; // ifade + atama
▶ if (toplam > 300) {         // koşul + blok
5     System.out.println("İndirim");
6 }
```

DEĞİŞKENLER

fiyat = 120

adet = 3

toplam = 360

ÇIKTI



İfade, Deyim ve Blok Adım Adım

Bellek (değişkenler)



Blok içindeki deyim çalışır: konsola **İndirim** yazılır.

Adım 6 / 7

```
1 int fiyat = 120;           // tanımlama deyimi
2 int adet = 3;              // tanımlama deyimi
3 int toplam = fiyat * adet; // ifade + atama
4 if (toplam > 300) {        // koşul + blok
    ▶ System.out.println("İndirim");
6 }
```

DEĞİŞKENLER

fiyat = 120

adet = 3

toplam = 360

ÇIKTI

İndirim

İfade, Deyim ve Blok Adım Adım

Bellek (değişkenler)



Blok biter. `adet = 2` olsaydı toplam 240 olur, koşul false olur ve blok hiç çalışmazdı.

Adım 7 / 7

```
1 int fiyat = 120;           // tanımlama deyimi
2 int adet = 3;              // tanımlama deyimi
3 int toplam = fiyat * adet; // ifade + atama
4 if (toplam > 300) {         // koşul + blok
5     System.out.println("İndirim");
6 }
```

DEĞİŞKENLER

fiyat = 120

adet = 3

toplam = 360

ÇIKTI

İndirim

Açıklamalar

- `//` satır sonuna kadar açıklamadır.
- `/* ... */` birden fazla satırı kapsar.
- `/** ... */` Javadoc (API belgesi) açıklamasıdır.
- Derleyici açıklamaları atlar; bytecode'a girmezler.
- Açıklama kodun ne yaptığını tekrarlamak yerine **nedenini** açıklamalıdır.

```
/** Merhaba programı (Javadoc). */  
public class Merhaba {  
    /* Birden fazla satıra  
       yayılan açıklama */  
    public static void main(String[] args) {  
        // selamı yaz  
        System.out.println("Merhaba");  
    }  
}
```

```
// selamı yaz  
System.out.println("Merhaba");  
/* açıklama */
```

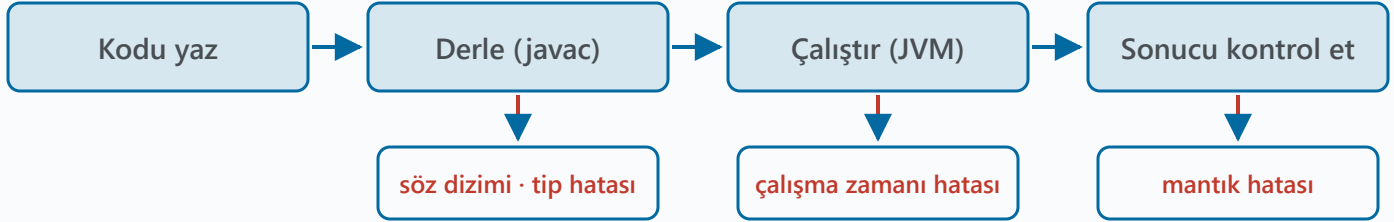
Merhaba.java (kaynak)

javac
→
açıklamaları atlar

```
getstatic System.out  
ldc "Merhaba"  
invokevirtual println  
return
```

Merhaba.class (bytecode): açıklama yok

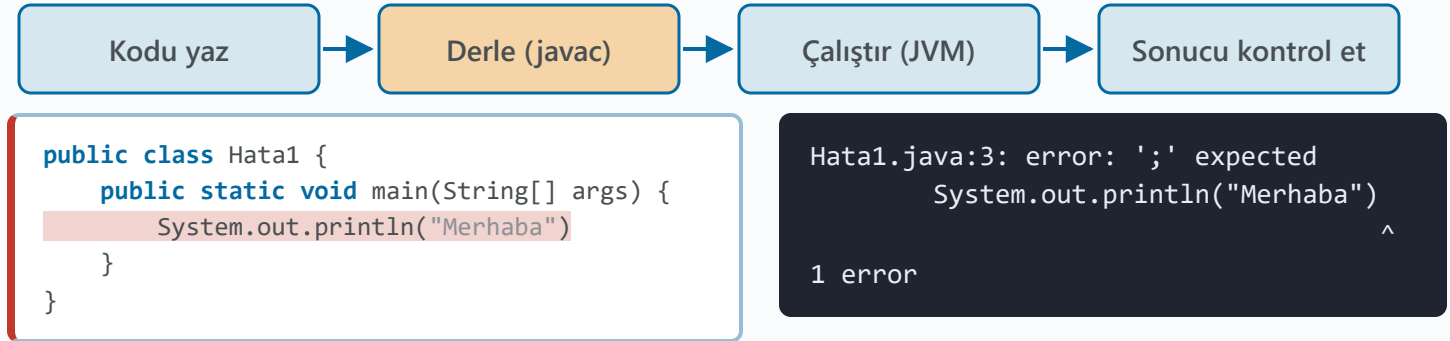
Hata Türleri



Hata	Ne zaman?	Örnek
Söz dizimi	Derleme sırasında	Eksik noktalı virgül
Tip	Derleme sırasında	Metni sayıya atama
Çalışma zamanı	Program çalışırken	Sıfıra bölme
Mantık	Program çalışır ama sonuç yanlıştır	Yanlış koşul

Hata Türleri Örneklerle

Söz dizimi hatası



Derleyici kuralı bozulan yeri satır numarasıyla gösterir; **.class üretilmez**. Düzeltme: satır sonuna `;` ekle.

Hata Türleri Örneklerle

Tip hatası

Kodu yaz

Derle (javac)

Çalıştır (JVM)

Sonucu kontrol et

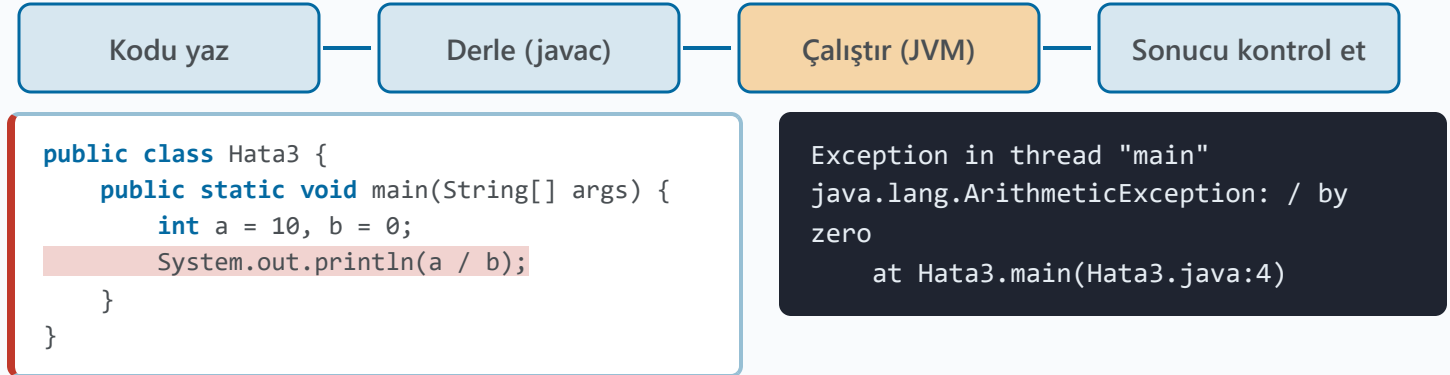
```
public class Hata2 {  
    public static void main(String[] args) {  
        int sayi = "42";  
    }  
}
```

```
Hata2.java:3: error: incompatible  
types:  
String cannot be converted to int  
    int sayi = "42";  
                ^
```

Dizgi (String) int kutusuna konamaz. Tipler derleme sırasında denetlenir. Düzeltme: `int sayi = 42;`

Hata Türleri Örneklerle

Çalışma zamanı hatası



Kod derlenir, ancak çalışırken tam sayı sıfıra bölünür: JVM bir **istisna** fırlatır ve program durur.

Hata Türleri Örneklerle

Mantık hatası

Kodu yaz

Derle (javac)

Çalıştır (JVM)

Sonucu kontrol et

```
public class Hata4 {  
    public static void main(String[] args) {  
        int a = 70, b = 90;  
        System.out.println(a + b / 2);  
    }  
}
```

115

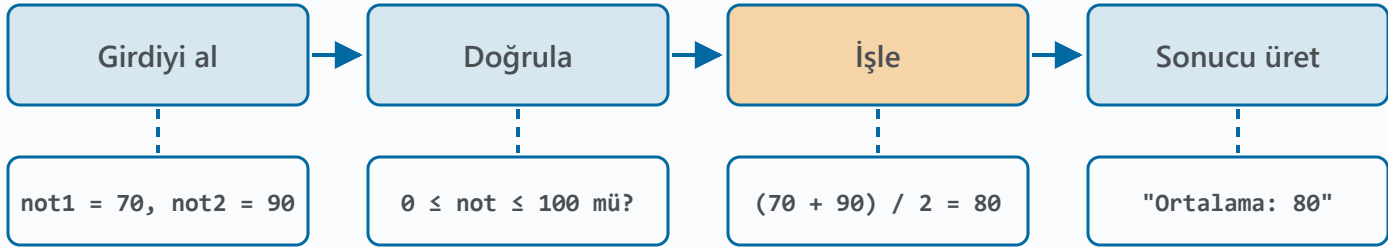
Hata iletisi yok, ama sonuç yanlış: beklenen 80. / önce yapılır: $70 + 45$. Düzeltme: $(a + b) / 2$.

Sık Derleme Hataları

Hata	Gerçek ileti	Çözüm
Noktalı virgül yok	<code>Merhaba.java:3: error: ';' expected</code>	önceki deyimin sonuna ; <code> ekle</code>
Metot adı yanlış (<code>System.out.println</code>)	<code>error: cannot find symbol symbol: method println(String) location: variable out of type PrintStream</code>	adı <code>println</code> yap
Küçük harf (<code>system.out</code>)	<code>error: package system does not exist</code>	<code>System</code> büyük S ile yazılır
Ad ile dosya uyuşmuyor	<code>error: class Selam is public, should be declared in a file named Selam.java</code>	dosya adı = public sınıf adı
Yanlış çalıştırma (<code>java Selam</code>)	<code>Error: Could not find or load main class Selam</code>	<code>.class</code> yok ya da ad/konum yanlış; uzantı yazma

İletiler JDK 21 ile alınmıştır; sürüme göre ek satırlar (ör. Caused by: ClassNotFoundException) çıkabilir. Her zaman ilk hatadan başla.

Program Bir Tariftir

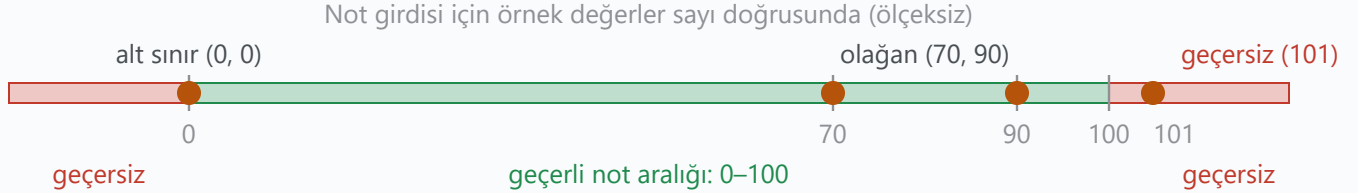


- Her adımın ön koşulu ve ürettiği değer belirlenmelidir.
- Büyük tarifler alt görevlere ayrılır (Java'da bunlar ileride **metot** olacak).

Tarifi kodlamadan önce elle sınavın: 70 ve 90 için adımları izleyince 80 çıkıyor mu? 101 girilirse hangi adım durdurur?

Kodlamadan Önce

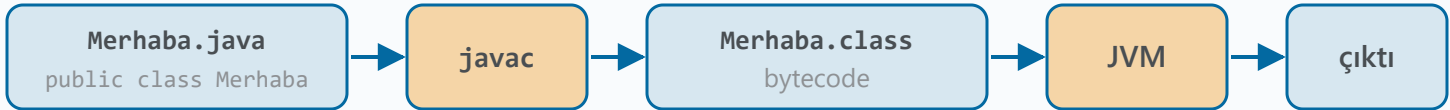
- Örnek girdi ve beklenen çıktıyı yazın.
- Çözümü doğal dil veya sözde kodla adımlara ayırın.
- Sınır durumlarını ve hata davranışını belirleyin.
- Ardından adımları Java yapılarına dönüştürün.



Örnek girdi	Beklenen çıktı	Neden?
70, 90	80	olağan durum
0, 0	0	alt sınır
101, 50	hata iletisi	geçersiz girdi

Bölüm Özeti

- Program, bilgisayarın bir görevi yerine getirmek için yürüttüğü komutlar bütünüdür.
- Java kaynak dosyası javac ile bytecode'a derlenir ve JVM üzerinde çalışır.
- public sınıf adı dosya adıyla eşleşir; main programın giriş noktasıdır.
- İfadeler değer üretir, deyimler işlemi tamamlar, bloklar deyimleri gruplar.
- Kodlamadan önce örnek girdi, beklenen çıktı ve sınır durumları belirlenir.



Dosya adı = public sınıf adı; JVM çalıştırmaya main metodundan başlar.

Kontrol Soruları

1. JDK, bytecode ve JVM arasındaki ilişki nedir?
2. Merhaba.java dosyası hangi komutlarla derlenip çalıştırılır?
3. main metodunun görevi nedir?
4. İfade ile deyim arasındaki fark nedir?
5. Söz dizimi hatası ile mantık hatası nasıl ayrılır?

Alıştırmalar

1. Adınızı, bölümünüzü ve öğrenci numaranızı üç ayrı satırda yazdıran `Kimlik` programını yazın, derleyin, çalıştırın.
2. Aynı çıktıyı tek bir `println` ve `\n` kullanarak üretin.
3. `\t` ile üç satırlık bir ders–kredi tablosu yazdırın.
4. `System.out.println("Merhaba")` satırındaki `;` işaretini silip derleyin; hata iletisinin satır numarasını ve `^` işaretini yorumlayın.
5. Sınıf adını `Merhaba`, dosya adını `merhaba.java` yapıp derleyin. Ne olur, neden?
6. `System.out.println(5 + 3);` ile `System.out.println("5 + 3");` çıktılarını tahmin edin; çalıştırarak doğrulayın.

Son

Java'ya Giriş ve İlk Program

Sonraki sunum: [Değişkenler, Veri Tipleri ve Operatörler](#) · [Çalışma soruları](#) · [Terim sözlüğü](#)